Software Architecture in Practice

A Scaffolded, Self-Directed Exercise for BSc (Hons) IT — Year 2

Case Study: "UniMart" — A Student Marketplace That Outgrows Itself

Compressed for a **single 3-hour workshop session**.

---

1. Purpose and Learning Outcomes

This exercise teaches architecture by making students **live through an architectural crisis**, not just read about one. They design a system as a monolith, watch it succeed and then start failing under real growth pressures, and must re-architect it — discovering the trade-offs themselves instead of being told about them.

By the end, students should be able to:

- Explain the difference between monolithic, layered, microservices, event-driven, and service-oriented architectures — in terms of forces, not just definitions.
- Identify which quality attributes (scalability, maintainability, deployability, fault isolation, time-to-market) drive a given architecture decision.
- Decompose a system into services using domain reasoning (bounded contexts), not arbitrary splitting.
- Produce standard architecture diagrams (context, container/component, deployment, sequence).
- Justify a trade-off in writing, to a non-architect stakeholder, using evidence from their own design work.

Format: Individual or pairs, single 3-hour session. Progressive stages — each stage's output feeds the next, so students cannot skip the reasoning and jump to "the answer." Timings are tight on purpose

Session Timeboxing (180 minutes)

Time	Stage	Activity
0:00–0:05	—	Downloading the document
0:05–0:20	1	Requirements & problem immersion
0:20–0:50	2	Design the monolith
0:50–1:05	3	Growth crisis — diagnose pain points
1:05–1:55	4	Redesign as microservices (core stage — most marks, most time)
1:55–2:15	5	Event-driven notification (quick pass)
2:15–2:35	6	Decision memo
2:35–2:50	—	2–3 pairs share their service boundaries + one trade-off with the room
2:50–3:00	—	Submission of the work
take-home	7	Optional build extension (not part of the 3 hrs)

2. Pre-Reading (Stage 0 — do before the session, ~15–20 min)

Short primer, not a lecture. Students should be able to answer these before starting:

- What is a "deployable unit," and why does it matter?
- What is coupling vs. cohesion, at the system level (not just class level)?
- What is a bounded context (in plain terms: "a part of the business with its own vocabulary and rules")?
- What do the "-ilities" mean: scalability, availability, maintainability, deployability?
- What is the C4 model (Context → Container → Component → Code) — students only need Context and Container levels for this exercise.

Suggested sources: Martin Fowler's writing on monoliths/microservices, and the C4 model site (c4model.com). Deliberately not prescribing exact links here — Search for "Fowler MonolithFirst" and "C4 model" and practice finding credible sources.

---

3. The Case Study

> **UniMart — Year 0 (the origin story)**
>
> Three final-year students at a mid-sized university built UniMart over one semester: a marketplace where students buy and sell used textbooks, electronics, and services (tutoring, printing, room cleaning) with other students on the same campus. It launched on one campus with about 2,000 active users.
>
> **Core functionality:**
>
> - Students register and log in (university email verification)
> - Sellers post listings (title, description, price, category, photos)
> - Buyers search/browse and filter listings
> - Buyers and sellers message each other to negotiate
> - Buyers pay through the app (card or campus wallet) and orders are tracked (posted → paid → completed)
> - Buyers leave ratings/reviews after a transaction
> - Users get notifications (new message, price drop, order status change)

> **Non-functional expectations at launch:** must work reliably during the two weeks at the start of each semester (textbook rush), payments must never be lost or double-charged, and the three founders — still students themselves — need to be able to build and ship it fast with almost no budget.

This is deliberately a compact but realistic domain: it has natural sub-domains (identity, catalog, messaging, payments/orders, reviews, notifications) without being so large students get lost.

---

4. Stage 1 — Requirements & Problem Immersion (15 min)

Task. Working only from the case study text above, produce — quickly, in bullet form, no polishing:

- A list of actors (types of users/systems interacting with UniMart).

- A list of functional modules implied by the narrative (don't invent features that aren't there).
- Three NFRs, each tied to a specific sentence in the case study (e.g., "must never double-charge" → consistency/reliability requirement on payments). Three is enough — don't chase completeness.
- A rough System Context diagram (C4 Level 1) sketch: one box for UniMart, boxes for external actors/systems (students, payment gateway, email/SMS provider), arrows for interactions. Whiteboard/paper sketch is fine — no internals yet.

---

5. Stage 2 — Design the Monolith (Year 0 architecture) (30 min)

Constraint to give students: *"You are the three founders. You have one semester, no budget, and no dedicated ops person. Design what you would actually build."*

Task. Design UniMart as a monolithic, layered architecture:

- Choose a layering scheme (e.g., Presentation → Business Logic → Data Access → Database) and sketch it.
- List the internal modules that live inside the single codebase (Auth, Catalog, Messaging, Orders/Payments, Reviews, Notifications) — same functional split as Stage 1, but now as packages/modules within one deployable, not separate services.
- Produce a quick deployment sketch: one application server, one database — where does it run (single VM / single container)?
- Justify the technology choice in 1–2 sentences — why is this the right call for this team, at this stage?

Reflection questions (answer in 1 sentence each — 3 sentences total):

- If a bug in the messaging feature crashes the server, what else goes down? Why?
- If textbook-rush traffic spikes hit only the search feature, what are your options for handling that load here? What's awkward about it?
- Roughly how long would a one-line fix to the reviews feature take to reach production, and why?

This stage should leave students with a design they're reasonably proud of — because for Year 0, it genuinely is the right choice. That's an important lesson in itself: monolith-first is often correct, not a mistake to avoid.

---

6. Stage 3 — The Growth Crisis (read, then diagnose) (15 min)

> **UniMart — 18 months later**
>
> UniMart is now live at 5 campuses with 60,000 users. The founding team has grown to 4 small squads: Catalog & Search, Messaging, Payments & Orders, and Platform (auth, notifications). Some specific problems have emerged:
>
> > • During the last textbook rush, a memory leak in the search feature brought down the entire application, including payments that were mid-transaction — some students were charged without receiving order confirmation.
> >
> > • The Payments squad wants to move to a specialised financial-transaction database with strict consistency guarantees. The Catalog squad wants to use a search-optimised database for fast filtering. In the current setup, everyone shares one database and one codebase, so neither team can make this change without affecting the other three squads.

> • A simple copy change to a listing page now takes 3 days to reach production, because it has to go through the same build, test, and deploy pipeline as the payments code — and the whole team is nervous about deploying payments code often.
> • Two squads have shipped conflicting changes to the same file in the same week, twice this month.

Task. Without redesigning anything yet, list each pain point and map it to the specific architectural cause in the Stage 2 monolith (shared process/failure domain, shared database, shared deploy pipeline, shared codebase/merge contention) — one line per pain point is enough. This mapping is the actual bridge from "monolith" to "why microservices exist" — do not skip straight to solutioning, even under time pressure.

---

7. Stage 4 — Redesign as Microservices (50 min)

This is the core stage — spend most of your remaining budget here.

(~10 min) Identify service boundaries using bounded-context reasoning — each service should map to one squad from the narrative and own its own data. Expected services: Auth/Identity, Catalog, Messaging, Orders & Payments, Reviews, Notifications. (Students may justify merging or splitting differently — the justification matters more than matching this exact list.)

(~15 min) For each service, specify — briefly, a few words per line is fine:

- Its single responsibility (one sentence)
- The data it owns (and only it can write to)
- 1–2 sample REST endpoints (method + path + purpose) — not a full API spec
- How it talks to other services — synchronous (REST/gRPC call) or asynchronous (event/message)? Justify the choice per interaction in one line (e.g., "Order confirmation → Notification: asynchronous, because Notification being briefly down shouldn't block a purchase").

(~15 min) Produce a C4 Container diagram: all services, an API gateway in front of them, each service's own datastore, arrows for communication (label sync vs async). One sequence diagram for the flow "Student buys a used textbook" (Catalog → Orders/Payments → Notification), if time allows — otherwise a verbal walkthrough of the flow is an acceptable substitute given the time limit.

(~10 min) Trade-off table. For at least these five dimensions, state what got better and what got harder compared to the monolith: deployability, fault isolation, data consistency, operational complexity, team autonomy. One line per dimension. Be specific to UniMart — generic statements ("microservices are more scalable") score poorly; grounded ones ("Catalog can now scale its search index independently of Payments during textbook rush") score well.

Note for students: microservices are not a strict upgrade. If your trade-off table doesn't include real costs (distributed transactions are hard; you now need service discovery, monitoring across services, and network failure handling), you've missed half the exercise.

---

8. Stage 5 — One More Pattern: Event-Driven Notification (20 min)

Task. Redesign just the notification flow using an event-driven approach:

- Introduce a message broker/event bus (name a real example: e.g., RabbitMQ, Kafka, or a cloud pub/sub service).
- Services publish events (OrderPlaced, NewMessageReceived, PriceDropped) without knowing who's listening.
- Notification subscribes to relevant events and acts independently.
- Quickly redraw the relevant slice of the container diagram to show publish/subscribe instead of direct calls.
- In 2–3 sentences: what does this buy you (decoupling, resilience to Notification being slow/down) and what does it cost (harder to trace "what happened and when," eventual consistency)?

Take-home extension *(not part of the 3-hour session):* research Service-Oriented Architecture (SOA) and Serverless/FaaS, and in one paragraph each, explain (a) how SOA differs from microservices (typically: heavier shared middleware/ESB, coarser-grained services) and (b) where a serverless function might fit UniMart (e.g., a nightly report-generation job that runs briefly and infrequently — a poor fit for a permanently-running service).

---

9. Stage 6 — Decision Memo (20 min)

Task. Write a 150–200 word memo addressed to "the UniMart co-founders," who are not architects, answering:

1. Was building a monolith at Year 0 the right call? Why?
2. What specifically triggered the need to change (cite the Stage 3 pain points, not generic theory)?
3. Is full microservices the right call at 60,000 users, or would something in between (e.g., splitting out just Payments, keeping the rest as a monolith — a "modular monolith" or partial decomposition) fit better? Defend a specific position.
4. What new problem should they expect to hit next, given the architecture you've now proposed?

This is the highest-value deliverable: it forces synthesis and shows whether the student actually understands why, not just what.

---

10. Optional Stretch — Build It (take-home, not part of the 3-hour session)

For students who want hands-on practice after the session: implement two of the services from Stage 4 (e.g., Catalog and Orders) as small, separately runnable applications (any stack — Node/Express, Flask, Spring Boot, etc.), each with its own local database, communicating over HTTP. Containerize each with Docker. The goal is simply to feel the difference — running two independent things, redeploying one without touching the other — not to build a production system.

---

11. Deliverables Checklist (In-Session, 3 hrs)

Stage	Time	Deliverable
1	15 min	Actor list, functional module list, 3 NFRs, rough Context diagram
2	30 min	Layered architecture sketch, module list, deployment sketch, tech justification, reflection answers

Stage	Time	Deliverable
3	15 min	Pain-point → architectural-cause mapping (one line each)
4	50 min	Service list with responsibilities/data/endpoints/communication, Container diagram, sequence diagram or walkthrough, trade-off table
5	20 min	Event-driven notification diagram + short reflection
6	20 min	Decision memo (150–200 words)
take-home	—	SOA/serverless paragraphs, optional two-service build

---

12. Assessment Rubric (suggested weighting, out of 100)

Criterion	Weight	What "good" looks like
Requirements grounding (Stage 1)	10%	NFRs traceable to specific case-study sentences, not invented
Monolith design quality (Stage 2)	15%	Coherent layering, honest reflection on its limits
Pain-point diagnosis (Stage 3)	10%	Each symptom correctly traced to a root architectural cause
Service decomposition reasoning (Stage 4)	25%	Boundaries follow business domains, not arbitrary splits; sync/async choices justified per-interaction
Diagram correctness & clarity (Stages 2, 4, 5)	15%	Follows C4 conventions, legible, consistent with written design
Trade-off depth (Stage 4–5)	15%	Specific to UniMart, includes real costs, not just benefits
Decision memo (Stage 6)	10%	Clear, evidence-based position; anticipates the next problem
Stretch prototype	Bonus (+5–10%)	Two services genuinely independently deployable